
Práctica 5: *TPB: Tu Primer Buscador*

1. Objetivo de esta práctica

El objetivo de esta práctica es resolver un problema concreto mediante la estrategia de Divide y Vencerás.

1.1. Problema a resolver

En el ámbito de las búsquedas de texto, es impracticable el intentar buscar cada vez en toda la colección de documentos (ej., imaginad un buscador Web que tuviera que acceder a toda las páginas web en tiempo real). Para acelerar el proceso, se utilizan todo tipo de índices que precálculan mucha información (todo tipo de frecuencias, temática, relevancia, ...) para que se puedan hacer las búsquedas en tiempo real. En esta práctica, vamos a implementar una versión muy simplificada de un índice inverso para acelerar las búsquedas en un conjunto de datos (*dataset*) de citas de autores.

Tenemos LISSS_2000, un dataset con 2000 citas de distintos autores y queremos hacer un programa que busque palabras tanto en el nombre del autor como en el contenido de las citas. En particular, tendréis que implementar tres búsquedas distintas:

- Búsqueda por autor: se dará una palabra y habrá que buscar qué citas han sido pronunciadas por cualquier autor que contenga la palabra en su nombre y apellidos.
- Búsqueda por palabra clave en la cita: se dará una palabra y se buscarán todas las citas que la contienen.
- Búsqueda multi-palabra en la cita: se dará una lista de palabras (en la línea de comandos) y se buscarán todas las citas que contienen **TODAS** las palabras (semántica AND).

Las búsquedas se harán todas en minúsculas, pero se respetarán los acentos (ej., “está” no será lo mismo que “esta”).

¿Qué es un Índice Invertido? Un índice invertido ¹ es una estructura de datos fundamental utilizada en motores de búsqueda y bases de datos para realizar búsquedas de texto completo de manera rápida y eficiente. A diferencia de un índice directo (que mapea un documento con las palabras que contiene), un índice invertido funciona al revés: mapea el contenido (como palabras, números o términos) a sus ubicaciones específicas dentro de un conjunto de documentos. En la práctica, actúa como el índice alfabético que se encuentra al final de un libro, donde cada palabra clave apunta a una lista de los identificadores de los documentos (o páginas) que la incluyen. Esto permite que, al

¹https://es.wikipedia.org/wiki/Índice_invertido

realizar una consulta, el sistema no tenga que escanear cada documento uno por uno, sino que simplemente busque el término en el índice y recupere instantáneamente todos los registros asociados.

Doc 1				Doc 2			
I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.				So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:			
term	docID	term	docID				
I	1	ambitious	2				
did	1	be	2				
enact	1	brutus	1				
julius	1	brutus	2				
caesar	1	capitol	1				
I	1	caesar	1				
was	1	caesar	2				
killed	1	caesar	2				
i'	1	did	1				
the	1	enact	1				
capitol	1	hath	1				
brutus	1	I	1				
killed	1	I	1				
me	1	i'	1				
so	2	it	2				
let	2	julius	1				
it	2	killed	1				
be	2	killed	1				
with	2	let	2				
caesar	2	me	1				
the	2	noble	2				
noble	2	so	2				
brutus	2	the	1				
hath	2	the	2				
told	2	told	2				
you	2	you	2				
caesar	2	was	1				
was	2	was	2				
ambitious	2	with	2				

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
me	1	→	1
noble	1	→	2
so	1	→	2
the	2	→	1 → 2
told	1	→	2
you	1	→	2
was	2	→	1 → 2
with	1	→	2

Figura 1: Ejemplo de un índice invertido dónde dos documentos en inglés (Doc1 y Doc2), se procesan para tener un índice dónde cada palabra apunta a qué documentos (id) la contienen (Ejemplo tomado de *An Introduction to Information Retrieval*, Manning et al. 2009).

Materiales Para facilitaros la tarea, se os entregan (aparte del dataset) dos ficheros con los índices invertidos ya creados. Los ficheros corresponden al índice invertido de palabras en las citas (LISSS_inv_idx_words.txt) y al índice invertido de autores (LISSS_inv_idx_auth.txt). Estos ficheros siguen un formato de campos separados por tabuladores (TSV):

palabra \t frecuencia \t < lista_documentos >

con `lista_documentos` siendo una lista de enteros separados por comas que denotan los identificadores de las citas que contienen la palabra de la línea. La frecuencia se tiene por completitud (pudiendo ser utilizada más adelante), así que guardadla en las estructuras de datos con las que trabajaréis.

El formato del dataset es también de campos separados por tabuladores (hay un `#` que tendréis que limpiar):

id \t cita \t # Nombre y apellidos del autor

Fijaos que tanto los índices invertidos, como el dataset de citas **están ordenados**. De esta manera, podréis leerlos adecuadamente y poder explotar ese orden en las búsquedas. Además, se os entregan unos `.hpp` con una sugerencia de estructuras de datos y de funciones/procedimientos base que son los mínimos para poder implementar la solución. Como detalle destacable, usaremos *Vector*, una clase de contenedores de la librería estándar de `c++` que nos permite tener vectores de tamaño variable de forma muy sencilla. Es importante que leáis la documentación de la clase, por ejemplo en <https://cplusplus.com/reference/vector/vector/>, para ver qué métodos nos proporciona, pero sobre todo haréis uso de:

- *push_back*: añade un elemento al final del vector.
- *size*: nos devuelve el tamaño actual del vector.
- *at*: devuelve el elemento en la posición *i-ésima* del vector comprobando el rango (lanza una excepción de tipo fuera de rango si no existe).

Como guía para la implementación:

- `dataset.hpp` contiene la definición de la estructura que va a mantener el fichero de citas cargado en memoria, de forma ordenada para poder acceder por posición. Serán nuestros datos.
- `indiceInvertido.hpp` contiene la definición de la estructura que vamos a tener para nuestros índices invertidos, basada en *vectorVarEnt* - vectores de enteros de tamaño variable, junto con las funciones y procedimientos necesarios para poder buscar, cargar, etc.
- `vectorVarEnt.hpp` es el tipo de datos de más bajo nivel que vamos a utilizar, será el almacén que utilizaremos en `indiceInvertido` para poder almacenar los índices de los documentos en dónde aparecen las palabras.

Estrategia a implementar En este caso (aunque existen métodos más rápidos para acceder), implementaremos las búsquedas mediante divide y vencerás. Como ejercicios *opcionales*, podéis comprobar dos cosas:

- Cómo se comportaría una búsqueda secuencial (tanto buscando citas como autores) sobre los índices.

- Cómo se comportaría una búsqueda secuencial sobre el dataset sin tener índices invertidos (lo notaréis sobre todo en la búsqueda multi-palabra).

La búsqueda multipalabra está pensada para que busquéis las palabras una por una y hagáis la intersección de las listas de documentos que devuelve cada búsqueda.

1.2. Requisitos del programa

Se deben implementar los tres tipos de búsquedas usando la estructura de datos que se proporciona mediante la estrategia de divide y vencerás. Para el desarrollo de la práctica, se debe tener en cuenta lo siguiente:

- El programa recibirá las opciones mediante línea de comandos.
- Las opciones serán:
 - “-a autor” para buscar el autor dado por el argumento *autor* en el índice de autores.
 - “-k keyword” para buscar la palabra dada por el argumento *keyword* en las citas.
 - “-mk [keywords]+” para buscar todas las palabras en las citas. El + en este caso significa una o más *keywords*.
- La salida no debe ser simplemente la lista de documentos, sino que hay que mostrar el id, la cita y el autor.

Ejemplos de ejecución Buscando por autor:

```
$ ./tpb -a einstein
1640 :: Albert Einstein
    Los ideales que iluminan mi camino y una y otra vez me han
    dado coraje para enfrentar la vida con alegría han sido: la
    amabilidad, la belleza y la verdad.
1418 :: Albert Einstein
    La alegría de ver y entender es el más perfecto don de la
    naturaleza.
1084 :: Albert Einstein
    El odio habita solo en los necios.
```

Buscando por keyword:

```
$ ./tpb -k música
1259 :: Beethoven
    La música debe hacer saltar fuego en el corazón del hombre,
    y lágrimas de los ojos de la mujer.
1795 :: Charles Dickens
    La dulce música de su voz y la alegría que reflejaba su
    hermoso rostro o el contacto de su mano, ejercían casi siempre
    sobre él una influencia beneficiosa.
60 :: Graham Greene
    A veces me pregunto cómo se las arreglan los que no escriben,
    los que no componen música o pintan, para escapar de la locura,
    de la melancolía, del terror pánico inherente a la condición
    humana.
```

Buscando por múltiples keywords:

```
$ ./tpb -mk música voz
1795 :: Charles Dickens
    La dulce música de su voz y la alegría que reflejaba su
    hermoso rostro o el contacto de su mano, ejercían casi siempre
    sobre él una influencia beneficiosa.
```

Fijaos en que las tildes sí que se tienen en cuenta (*música*), en que se busca en minúsculas y en que, al añadir una palabra extra a la búsqueda, al ser semántica AND, los resultados tienen que ser un subconjunto estricto de las dos por separado (i.e., si buscas *música* y *voz* por separado, ambas búsquedas te darán resultados más amplios).

1.3. Fichero Makefile

Se debe escribir un fichero **Makefile** que permita compilar el programa **tpb** mediante la orden:

```
$ make
```

2. Resultados del trabajo desarrollado en las cinco prácticas de la asignatura

Como resultado de las prácticas, cada alumno dispondrá en su cuenta de un directorio (carpeta) denominado **programacion2** dentro del cual se podrán encontrar los directorios (carpetas) y ficheros que se detallan a continuación.

- Carpeta **programacion2/funciones** con los siguientes ficheros: **pilaEnt.hpp** y **pilaEnt.cpp**
- Carpeta **programacion2/practica1**, con los siguientes ficheros: **tiempoReaccion.cpp**, **generarTabla.cpp**, **genNum.hpp**, **genNum.cpp** y **aproxPI.cpp**.
- Carpeta **programacion2/practica2** con los siguientes ficheros: **calculos.hpp**, **calculos.cpp**, **pruebasCal.cpp** y **Make.pruebasCal**

- Carpeta **programacion2/practica3** con los siguientes ficheros:
 - Ficheros de interfaz y de implementación, **funcionesPilaEnt.hpp** y **funcionesPilaEnt.cpp**.
 - Fichero **pruebasPila.cpp** con el programa de prueba.
 - Fichero **Makefile** para compilar el programa de prueba.
- Carpeta **programacion2/practica4** con los siguientes ficheros:
 - Ficheros de interfaz y de implementación, **laberinto.hpp** y **laberinto.cpp**.
 - Fichero de implementación **mainLab.cpp**.
 - Fichero **Makefile** para obtener el ejecutable **mainLab**.
- Carpeta **programacion2/practica5** con los siguientes ficheros:
 - Fichero **tpb.cpp** que contiene las funciones necesarias para implementar la funcionalidad propuesta de acuerdo a los requisitos establecidos.
 - Ficheros **dataset.hpp**, **dataset.cpp**, **indiceInvertido.hpp**, **indiceInvertido.cpp**, **vectorVarEnt.hpp**, y **vectorVarEnt.cpp** con las implementaciones propuestas.
 - Fichero **Makefile** para compilar **tpb.cpp**.